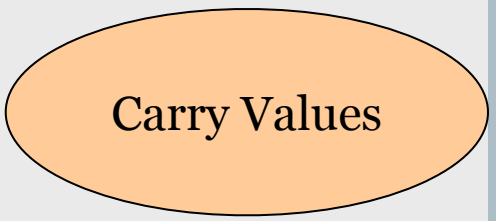
CSCI 1101 Introduction to Computer Science

ARITHMETIC IN NON-DECIMAL BASES AND REPRESENTING INTEGER VALUES

Arithmetic in Binary

Remember that there are only 2 digits in binary, 0 and 1



			1	1	1	1	1	
		1	0	1	0	1	1	1
+		1	0	0	1	0	1	1
	1	0	1	0	0	0	1	0*

*: 1+1 = 2
No digit for 2 in Binary. So 2-2=**0** and a **carry**

Subtracting Binary Numbers

Remember borrowing? Apply that concept here:

	~~1~~ ^{0}	~~0~~ ^{2}	~~1~~ ^{2}	0 ^{2}	1	1	1
		~~0~~ $^{-1}$	~~1~~ ^{0}				
-		1	1	1	0	1	1
	0	0	1	1	1	0	0

If minuend < subtrahend borrow **2** from immediate left column and subtract **1** from value in that column.

Same idea, different base (hex)

- Addition

$$\begin{array}{cccc} & & 1 & \\ & & A & 3 \\ + & & 1 & E \\ \hline & & C & 1^{*} \end{array}$$

*: 3+E = 3+14=17
No digit for 17 in Hex
So 17-16=1 and a **carry**

- Subtraction

$$\begin{array}{cccc} & & A^{9} & 3^{16} \\ - & & 1 & E \\ \hline & & 8 & 5 \end{array}$$

If minuend < subtrahend borrow **16** from immediate left column and subtract **1** from value in that column.

Analog and Digital Information

Computers are finite!

How do we represent an infinite world?

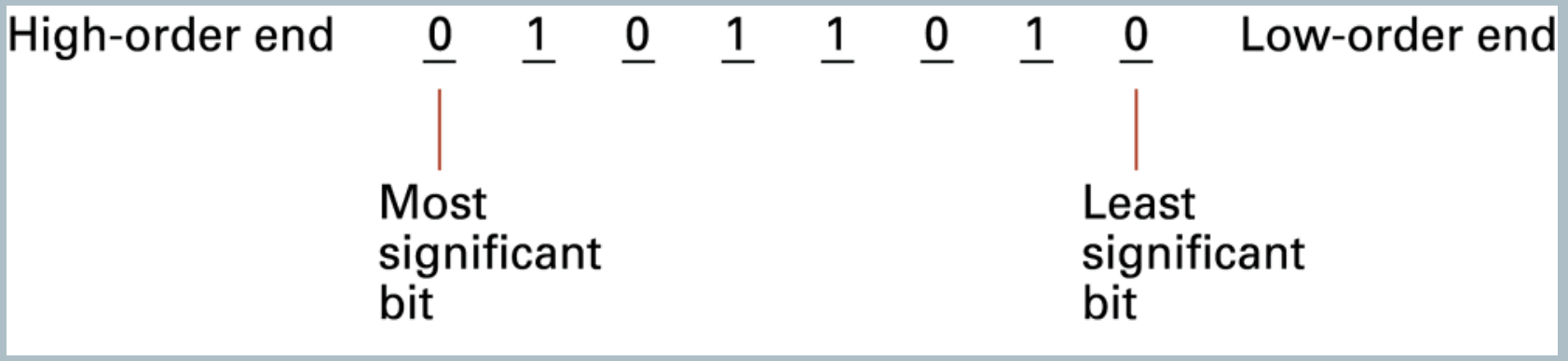
We represent enough of the world to satisfy our computational needs and our senses of sight and sound

Bits and Bit Patterns

- **Bit:** Binary Digit (0 or 1)
- Bit Patterns are used to represent information.
  - Numbers
  - Text characters
  - Images
  - Sound
  - And others

Main Memory Cells

- **Cell:** A unit of main memory (typically 8 bits which is one **byte**)
  - **Most significant bit:** the bit at the left (high-order) end of the conceptual row of bits in a memory cell
  - **Least significant bit:** the bit at the right (low-order) end of the conceptual row of bits in a memory cell



Binary Representations

One bit can be either 0 or 1

One bit can represent two things *(Why?)*

Two bits can represent four things *(Why?)*

How many things can three bits represent?

How many things can four bits represent?

How many things can eight bits represent?

Binary Representations

1 Bit	2 Bits	3 Bits	4 Bits	5 Bits
0	00	000	0000	00000
1	01	001	0001	00001
	10	010	0010	00010
	11	011	0011	00011
		100	0100	00100
		101	0101	00101
		110	0110	00110
		111	0111	00111
			1000	01000
			1001	01001
			1010	01010
			1011	01011
			1100	01100
			1101	01101
			1110	01110
			1111	01111
				10000
				10001
				10010
				10011
				10100
				10101
				10110
				10111
				11000
				11001
				11010
				11011
				11100
				11101
				11110
				11111

Counting with binary bits

Binary Representations

How many things can η bits *represent?*

Why?

What happens every time you increase the number of bits by one?

Representing Numeric Data

Quick Note:

Integers and floating-point (real) numbers are different

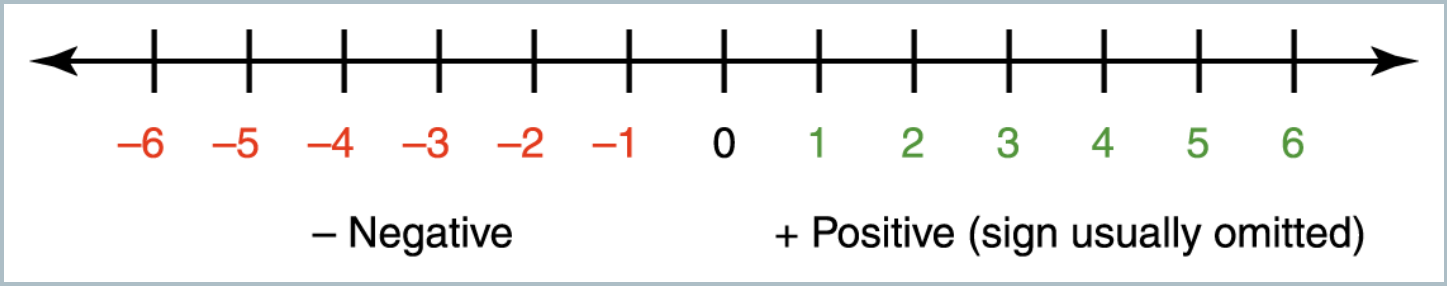
These kinds of values are represented differently in computers.

Integers first…

Representing Negative Values

Signed-magnitude number representation

- The sign represents the ordering, and the digits represent the **magnitude** of the number.
- This is what we use for decimal numbers.



Representing Negative Values

- There is a problem with the sign-magnitude representation: every number should have a positive and a negative representation and so there is a plus zero and minus zero

- We make an exception with the zero and ignore the minus zero but this causes unnecessary complexity to the hardware of a computer

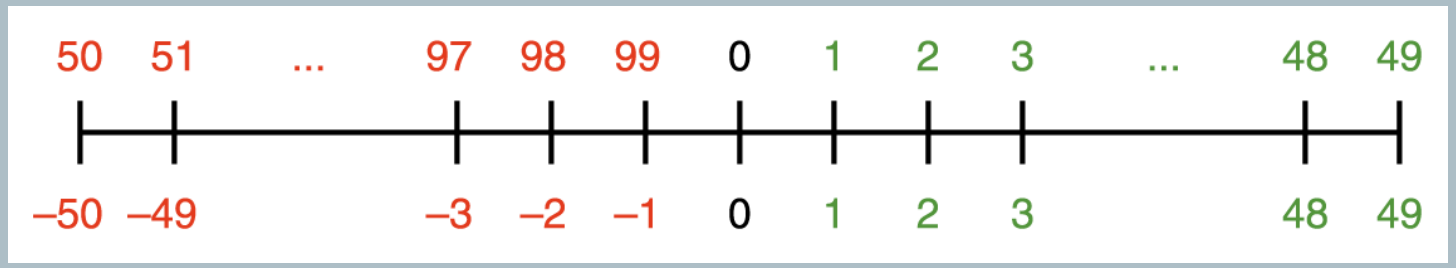
- Solution:

  Keep all numbers as integer values, with half of them representing negative numbers. Here's how…

Representing Negative Values

- First, think about it using decimal numbers.
- Using two decimal digits
  - let 1 through 49 represent 1 through 49 (positive values)
  - let 50 through 99 represent -50 through -1 (negative values)
- This representation is known as **10's complement**.

Notice that only negative numbers have a "different" representation in both systems (signed magnitude and 10's complement).



Representing Negative Values

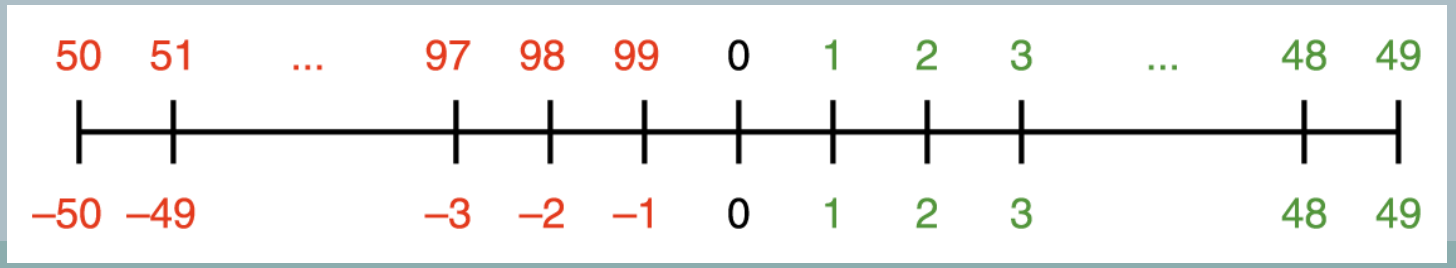
To perform addition, add the numbers and discard any carry

Signed-Magnitude	New Scheme
5 + −6 −1	5 + 94 99
−4 + 6 2	96 + 6 2
−2 + −4 −6	98 + 96 94

Now you try it

48 (signed-magnitude)
-1
47

How does it work in the new scheme?



Representing Negative Values

A-B=A+(-B)

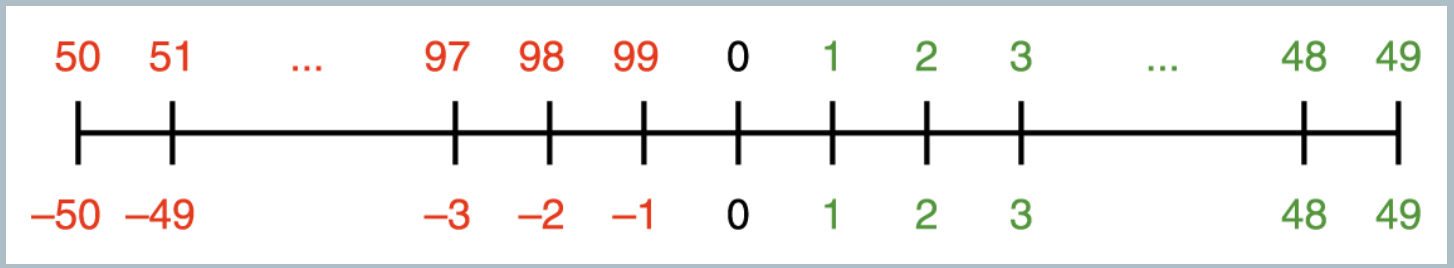
Add the negative of the second to the first

Signed-Magnitude	New Scheme	Add Negative
–5	95	95
– 3	– 3	+ 97
–8		92

Try

4 - 4 -4

- 3 ±3 ± -3



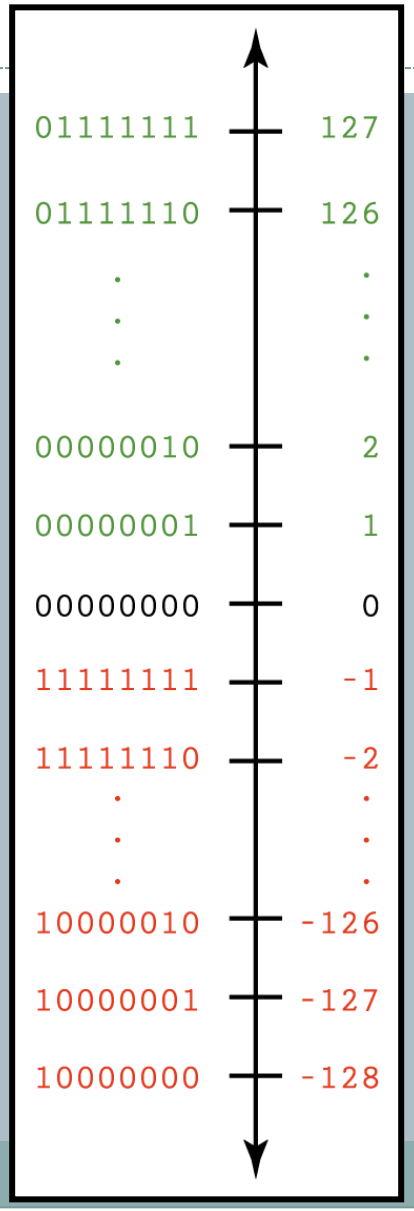
Representing Negative Values

- Now for binary
- **Two's Complement**

(Vertical line is easier to read)

- Notice that all positive numbers have a zero in the most significant bit while all negative numbers have a one in the most significant bit

(this is a quick way to determine if a number is positive or negative in the 2's complement representation)



Finding the Two's complement representation

- **IMPORTANT: The idea of the representation and the algorithm (procedure) for finding the representation are distinct from each other!**
- Positive values are simply the "regular" binary
- To find the two's complement representation of a **negative** number:

1. Flip the bits of the positive binary of the magnitude
2. Add 1

Or use this alternative method:

http://www.youtube.com/watch?v=zWWWZJ_w2CA

Representing Negative Values

Addition and subtraction are the same as in 10's complement arithmetic

-127	10000001
+ 1	00000001
-126	10000010

Notice the left-most bit is a one (1) indicating a negative number.

Number Overflow

What happens if the computed value won't fit?

Overflow Watch this video for a nice explanation:
http://www.youtube.com/watch?v=rnQwucEfT6A

If each value is stored using 8 bits, adding 127 to 3 **overflows**

	1	1	1	1	1	1	1	
	0	1	1	1	1	1	1	1
+	0	0	0	0	0	0	1	1
	1	0	0	0	0	0	1	0

Problems occur when mapping an infinite world onto a finite machine!

Number Overflow

Important things to remember when adding numbers in 2's complement notation to determine if there is **overflow**:

1) You must know the length of the number (number of bits)
2) You must disregard any bit you may get as a carry on the leftmost column (most significant bit)
3) If the result of adding 2 positive numbers is negative or the result of adding 2 negative numbers is positive then there is OVERFLOW
4) Adding a negative to a positive number or vice versa will never produce an overflow because they tend to cancel each other

None of the examples below produce overflow:

```
1 1 0 0 0 0 0 0
  0 1 0 0 1 0 0 1
+ 1 1 1 1 0 1 0 0
-----------------
± 0 0 1 1 1 1 0 1
```

No overflow!!!

```
1 1 0 1 1 1 0 1
  1 1 0 0 0 1 0 1
+ 1 1 0 1 1 1 0 1
-----------------
± 1 0 1 0 0 0 1 0
```

Representing Real Numbers

Real numbers in the decimal system

A number with a whole part and a fractional part

104.32, 0.999999, 357.0, and 3.14159

Weights (place values) to the right of the decimal point have negative exponents and the base is 10:

10^{-1}, 10^{-2} , 10^{-3} , 10^{-4}, …

Representing Real Numbers

Real numbers in the binary system

A number with a whole part and a fractional part

10.101, 0.11101, 1011.1, and 11.111

Weights (place values) to the right of the radix point have negative exponents and the base is 2:

2^{-1}, 2^{-2}, 2^{-3}, 2^{-4}, …

Converting **binary** fractions to **decimal** fractions

- Multiply each fractional bit by 2 raised to the corresponding power
- Solve the sum

Example

$(0.101)_2 = (\quad)_{10}$

$1 * 2^{-1} + 0 * 2^{-2} + 1 * 2^{-3}$

This is equivalent to

$1 * (1/2) + 0 * (1/4) + 1 * (1/8) = .625$

Thus: 0.101 = 0.625

Converting **decimal** fractions to **binary** fractions

- Multiply the fraction by 2
- The whole part is the binary digit
- Take the fractional part and repeat the steps above until the fractional part is zero or you have enough binary digits

Example

$(0.832)_{10} = (\quad)_2$

.832 * 2 = 1.664 => 1
.664 * 2 = 1.328 => 1
.328 * 2 = 0.656 => 0
.656 * 2 = 1.312 => 1

Thus, $(0.832)_{10} \approx (0.1101)_2$

Representing Real Numbers

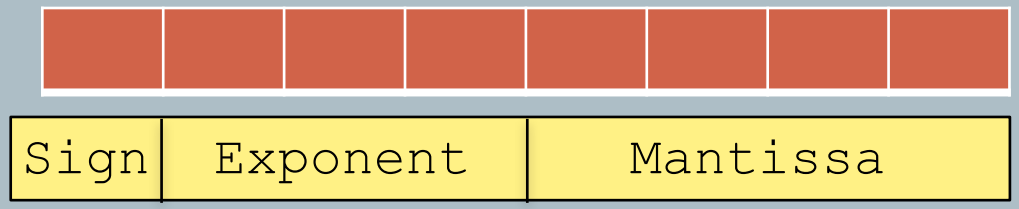
A real value in base 10 can be defined by the following formula

$$\text{sign} * \text{mantissa} * 10^{\text{exp}}$$

The representation is called **floating point** because the number of digits is fixed but the radix point floats

Storing Real Numbers

- A binary representation has to account for all the "parts" of the floating point representation
  - Sign: 0 to indicate positive, 1 to indicate negative
  - Mantissa: the bits that result from normalizing the number (the radix point is moved to the left of the leftmost 1 in the number)
  - Exponent: indicates the number of positions the radix point needs to be moved to the right (if the exponent is positive) or to the left (if the exponent is negative) to go back to its original location

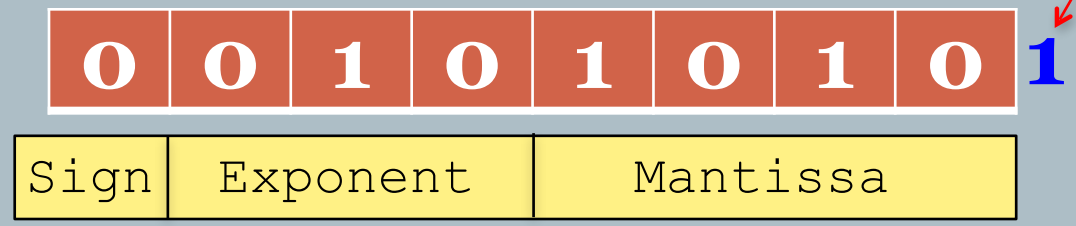


Storing Real Numbers

Example: $(2.625)_{10} = (10.101)_2$

Normalized mantissa = .10101

Exponent = 010 (+2 in base 10)



What if there isn't enough room for some part of the representation (notably the mantissa)?

- **Truncation error**, *aka* floating-point error.

Real world example

Microsoft Visual C++ is consistent with the IEEE numeric standards: it stores real numbers according to the table shown below.

Value	Stored as
Float	sign bit, 8-bit exponent, 23-bit mantissa
Double	sign bit, 11-bit exponent, 52-bit mantissa